Hybrid KEM Constructions from Classical PKEs and Post-Quantum KEMs

Biming Zhou[1], Yukai Zhang[1], Haodong Jiang[2], and Yunlei Zhao[1]

[1] Fudan University, Shanghai 200433, China
[2] Henan Key Laboratory of Network Cryptography Technology, Zhengzhou, 450001, Henan, China
bmzhou22@m.fudan.edu.cn, 25113050312@m.fudan.edu.cn, hdjiang13@163.com, ylzhao@fudan.edu.cn

Abstract. The rapid progress of quantum computing threatens widely deployed public-key cryptosystems such as RSA and Diffie–Hellman, accelerating the transition toward post-quantum cryptography (PQC). During this migration, hybrid key encapsulation mechanisms (KEMs) that combine classical and post-quantum primitives are strongly recommended by standardization bodies and cybersecurity agencies. However, existing hybrid designs mainly focus on combining post-quantum KEMs with Diffie–Hellman–style constructions, while the systematic integration of standardized classical public-key encryption (PKE) schemes with post-quantum KEMs remains largely unexplored.

In this work, we introduce two generic hybrid constructions, HybKEM and HybKEM*, that combine a classical PKE scheme with a post-quantum KEM satisfying ciphertext second-preimage resistance (C2PRI). We prove that both constructions achieve IND-CCA security in the standard model. The refined construction HybKEM* additionally relies on a new security notion of the classical PKE scheme, termed partial ciphertext second-preimage resistance (PC2PRI), which captures second-preimage resistance when a designated ciphertext component is fixed. This new property enables shared-key derivation from only a designated PKE ciphertext component in HybKEM*, leading to improved efficiency. Finally, we provide a systematic analysis of the PC2PRI property for several standardized classical encryption schemes, including ECIES, PSEC, and SM2.

1 Introduction

As quantum computing advances, widely deployed public-key cryptosystems such as RSA [29] and Diffie–Hellman [20] face fundamental threats due to Shor's algorithm [30]. This motivates the transition to post-quantum cryptography (PQC) in order to ensure security against quantum adversaries.

The key encapsulation mechanism (KEM) is a fundamental cryptographic primitive for establishing shared secrets between two parties. To balance security and interoperability during the transition to post-quantum cryptography (PQC), national cybersecurity agencies [2, 3, 28] recommend the deployment of hybrid

KEM constructions that combine classical and post-quantum algorithms. Such hybrid designs ensure that the overall scheme remains secure even if one of the underlying components is compromised. The motivation for hybrid KEMs stems from the fact that post-quantum schemes are relatively new and may still lack extensive cryptanalysis and mature implementations. In contrast, classical public-key primitives have been studied for decades and can therefore serve as a conservative hedge against potential weaknesses in emerging post-quantum constructions.

Several hybrid KEM constructions have been proposed in the literature [7,9, 15,16,23,27], primarily focusing on combining post-quantum KEMs with classical Diffie–Hellman (DH) key agreement, such as X25519 [13,26]. A notable line of work [7,9,15,16] develops more efficient hybrid constructions by eliminating redundant inputs in PRF computations. More recently, Liu et al. [27] propose an efficient hybrid KEM that directly combines a post-quantum public-key encryption scheme with DH, further reducing redundant hash evaluations. In contrast, the systematic integration of standardized classical public-key encryption (PKE) schemes such as ECIES [25], PSEC [31], RSA-OAEP [12,22], and SM2 [32] with post-quantum KEMs remains largely unexplored.

1.1 Our Contributions

In this work, we present a systematic framework for hybrid KEM constructions from post-quantum KEMs and classical public-key encryption schemes. Our main contributions are as follows.

- We introduce a new security notion for public-key encryption, termed *partial ciphertext second-preimage resistance* (PC2PRI). This notion extends the ciphertext second-preimage resistance (C2PRI) property for KEMs [9] to the PKE setting. It captures second-preimage resistance when a designated ciphertext component is fixed. We further provide a systematic study of the PC2PRI property for several standardized classical encryption schemes, including ECIES, PSEC, and SM2.
- We propose two generic hybrid KEM constructions, denoted HybKEM and HybKEM*. The scheme HybKEM combines a post-quantum KEM satisfying C2PRI with a classical PKE scheme and derives the shared key from the full PKE ciphertext. The more efficient variant HybKEM* additionally requires the PKE scheme to satisfy PC2PRI, enabling shared key derivation from only a designated PKE ciphertext component. We prove that both constructions achieve IND-CCA security in the standard model.

1.2 Related Work

In independent and concurrent work [17], the authors propose *StarHunters*, a general framework for constructing hybrid KEMs from a classical PKE scheme and a post-quantum KEM. Their approach follows a two-layer paradigm. First, they present a generic transformation that converts an IND-CCA-secure PKE

scheme into an IND-CCA-secure KEM. They then design a composition framework that combines two component KEMs to obtain an IND-CCA-secure hybrid KEM.

In contrast, we propose two hybrid constructions, HybKEM and HybKEM*, directly from a classical PKE scheme with a post-quantum KEM. Furthermore, we introduce and formalize the PC2PRI property for classical PKE schemes and show that this property enables a more efficient hybrid design in HybKEM*.

2 Preliminaries

2.1 Notation

The security parameter is denoted as λ. PPT is denoted to represent probabilistic polynomial time. QPT is denoted to represent quantum polynomial time. $\mathcal{K}$, $\mathcal{M}$ and $\mathcal{C}$ denote the key space, message space, and ciphertext space, respectively. For a finite set X, $x \leftarrow\!\!\$\ X$ represents the sampling of a uniformly random element from X. For a randomized classical algorithm (resp. deterministic) A, $y \leftarrow\!\!\$\ A(x)$ (resp. $y \leftarrow A(x)$) denotes that A outputs y on input x. $x =?y$ is denoted as an integer that is 1 if $x = y$, and 0 otherwise. $|X|$ denotes the cardinality of set X.

2.2 Cryptographic Primitives

Definition 1 (Public-Key Encryption). *A public-key encryption (PKE) scheme over message space $\mathcal{M}$ is a tuple of PPT algorithms* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *defined as follows.*

- $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}(1^\lambda)$*: the key generation algorithm* KeyGen *takes as input the security parameter and outputs a key pair* $(\mathsf{pk}, \mathsf{sk})$*. Usually, we will omit the input of* KeyGen *for brevity.*
- $c \leftarrow\!\!\$\ \mathsf{Enc}(\mathsf{pk}, m)$*: the encryption algorithm takes a public key* pk *and a message $m \in \mathcal{M}$ and outputs a ciphertext c.*
- $m' \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$*: the decryption algorithm takes the secret key* sk *and a ciphertext c and outputs a message $m' \in \mathcal{M} \cup \{\perp\}$.*

Correctness. A PKE scheme is δ-correct if

$$\mathbf{E}_{(\mathsf{pk},\mathsf{sk})\leftarrow\!\!\$\mathsf{KeyGen}()}\left[\max_{m\in\mathcal{M}} \Pr[\mathsf{Dec}(\mathsf{sk}, c) \neq m : c \leftarrow \mathsf{Enc}(\mathsf{pk}, m)]\right] \leq \delta.$$

We say the scheme is perfectly correct if $\delta = 0$.

Definition 2 (IND-CCA Security for PKE). *We define the* IND-CCA *game as shown in Fig. 1, and the* IND-CCA *advantage function of an adversary $\mathcal{A}$ against PKE as:*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{PKE}}(\mathcal{A}) := \left|\Pr[\mathsf{IND\text{-}CCA}^{\mathcal{A}}_{\mathsf{PKE}} = 1] - \frac{1}{2}\right|.$$

$\mathsf{IND\text{-}CCA}^{\mathcal{A}}_{\mathsf{PKE}}$	$\mathsf{O}^{\mathsf{Dec}}(c)$
$(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}()$	**if** $c = c^*$ **return** $\perp$
$b \leftarrow\!\!\$\ \{0,1\}$	**return** $\mathsf{Dec}(\mathsf{sk}, c)$
$(m_0, m_1) \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Dec}}(\cdot)}(\mathsf{pk})$	
$c^* \leftarrow\!\!\$\ \mathsf{Enc}(\mathsf{pk}, m_b)$	
$b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Dec}}}(\mathsf{pk}, c^*)$	
return $b' =? b$	

Fig. 1: IND-CCA security game for PKE $\mathsf{PKE} := (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$.

Definition 3 (Key Encapsulation Mechanism). *A key encapsulation mechanism (KEM) with key space $\mathcal{K}$ is a tuple of PPT algorithms* KeyGen*,* Encaps*,* Decaps *defined as follows.*

- $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}(1^\lambda)$*: The key generation algorithm* KeyGen *takes as input the security parameter and outputs a key pair* $(\mathsf{pk}, \mathsf{sk})$*. Usually, we will omit the input of* KeyGen *for brevity.*
- $(c, K) \leftarrow\!\!\$\ \mathsf{Encaps}(\mathsf{pk})$*: The encapsulation algorithm* Encaps *takes as input the public key* pk *and outputs a tuple (c, K), where $K \in \mathcal{K}$.*
- $K' \leftarrow \mathsf{Decaps}(\mathsf{sk}, c)$*: The decapsulation procedure, on input the secret key* sk *and the ciphertext c, outputs a key $K' \in \mathcal{K}$ or the rejection symbol $\perp \notin \mathcal{K}$.*

Correctness. A KEM scheme is said to be δ-correct if

$$\Pr\left[\mathsf{Decaps}(\mathsf{sk}, c) \neq K \;\middle|\; (\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}(1^\lambda);\ (c, K) \leftarrow\!\!\$\ \mathsf{Encaps}(\mathsf{pk})\right] \leq \delta.$$

$\mathsf{IND\text{-}CCA}^{\mathcal{A}}_{\mathsf{KEM}}$	$\mathsf{O}^{\mathsf{Decaps}}(c)$
$(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}()$	**if** $c = c^*$: **return** $\perp$
$b \leftarrow\!\!\$\ \{0,1\}$	**return** $K := \mathsf{Decaps}(\mathsf{sk}, c)$
$(c^*, K_0^*) \leftarrow\!\!\$\ \mathsf{Encaps}(\mathsf{pk})$	
$K_1^* \leftarrow\!\!\$\ \mathcal{K}$	
$b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Decaps}}}(\mathsf{pk}, c^*, K_b^*)$	
return $b' =? b$	

Fig. 2: IND-CCA security game for KEM $\mathsf{KEM} := (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$.

Definition 4 (IND-CCA Security for KEM). *We define the* IND-CCA *game as shown in Fig. 2, and the* IND-CCA *advantage function of an adversary*

$\mathcal{A}$ *against KEM as:*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{A}) := \left|\Pr[\mathsf{IND\text{-}CCA}^{\mathcal{A}}_{\mathsf{KEM}} = 1] - \frac{1}{2}\right|.$$

Definition 5 (Pseudorandom Function (PRF)). *Let $\mathcal{K}$ be a finite key space, $\mathcal{X}$ an input space, and $\mathcal{Y}$ a finite output space. A pseudorandom function (PRF) is an efficiently computable keyed deterministic function $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$. We define the* PRF *game as shown in Fig. 3, and the PRF advantage of adversary $\mathcal{A}$ is defined as*

$$\mathsf{Adv}^{\mathsf{PRF}}_{F}(\mathcal{A}) := \left|\Pr[\mathsf{PRF}^{F}_{0}(\mathcal{A}) \Rightarrow 1] - \Pr[\mathsf{PRF}^{F}_{1}(\mathcal{A}) \Rightarrow 1]\right|.$$

$\mathsf{PRF}^{F}_{b}(\mathcal{A})$		$\mathsf{Eval}_{F}(x)$	
1 :	$\mathbf{T}[\cdot] \leftarrow \emptyset$	1 :	**if** $x \in \mathbf{T}$ **return** $\mathbf{T}[x]$
2 :	$k \leftarrow\!\!\$\ \mathcal{K}$	2 :	$y_0 \leftarrow F(k, x)$
3 :	$b' \leftarrow \mathcal{A}^{\mathrm{Eval}_F(\cdot)}$	3 :	$y_1 \leftarrow\!\!\$\ \mathcal{Y}$
4 :	**return** b'	4 :	$\mathbf{T}[x] \leftarrow y_b$
		5 :	**return** y_b

Fig. 3: PRF security games.

Definition 6 (Message Authentication Code (MAC)). *A message authentication code (MAC) scheme with tag space $\mathcal{T}_{\mathsf{MAC}}$ and key space $\mathcal{K}_{\mathsf{MAC}}$ is a tuple of PPT algorithms* $(\mathsf{Mac}, \mathsf{MacVf})$ *defined as follows.*

- $\mathsf{tag} \leftarrow \mathsf{Mac}(k, m)$*: The tagging algorithm takes as input a key $k \in \mathcal{K}_{\mathsf{MAC}}$ and a message $m \in \{0,1\}^*$, and outputs a tag $\mathsf{tag} \in \mathcal{T}_{\mathsf{MAC}}$.*
- $b \leftarrow \mathsf{MacVf}(k, m, \mathsf{tag})$*: The verification algorithm takes as input a key $k \in \mathcal{K}_{\mathsf{MAC}}$, a message $m \in \{0,1\}^*$, and a tag $\mathsf{tag} \in \mathcal{T}_{\mathsf{MAC}}$, and outputs a bit $b \in \{0,1\}$ indicating acceptance or rejection.*

Correctness. A MAC scheme is correct if for all keys $k \in \mathcal{K}_{\mathsf{MAC}}$ and all messages $m \in \{0,1\}^*$,

$$\Pr[\mathsf{MacVf}(k, m, \mathsf{Mac}(k, m)) = 1] = 1.$$

For deterministic MAC schemes, verification can be equivalently performed by recomputing the tag and checking for equality. Next, we define the MacColl security notion for MAC schemes. It is easy to see that MacColl security is achieved when $\mathsf{Mac}(k, m)$ is defined as $\mathsf{H}(k \parallel m)$ for any collision-resistant hash function H.

Definition 7 (MacColl Security for MAC [10]). *The* MacColl advantage *of an adversary* $\mathcal{A}$ *against a MAC scheme* MAC *is defined as*

$$\mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{A}) = \Pr\left[\begin{array}{l}(k_1, m_1) \neq (k_2, m_2)\\ \wedge\ \mathsf{MacVf}(k_1, m_1, \mathsf{tag}) = 1\\ \wedge\ \mathsf{MacVf}(k_2, m_2, \mathsf{tag}) = 1\end{array} : (m_1, m_2, k_1, k_2, \mathsf{tag}) \leftarrow \mathcal{A}()\right].$$

Definition 8 (Key Derivation Function). *A key derivation function is a deterministic polynomial-time algorithm* $\mathsf{KDF} : \mathcal{Z} \to \mathcal{K}$ *that maps a shared secret* Z *to a key* $k \in \mathcal{K}$.

Definition 9 (Collision Resistance of KDF). *The collision advantage of an adversary* $\mathcal{A}$ *is*

$$\mathsf{Adv}^{\mathsf{Coll}}_{\mathsf{KDF}}(\mathcal{A}) := \Pr\left[Z \neq Z' \wedge\ \mathsf{KDF}(Z) = \mathsf{KDF}(Z') : (Z, Z') \leftarrow \mathcal{A}\right].$$

2.3 (Partial) Ciphertext Second-Preimage Resistance

We first recall the notion of ciphertext second-preimage resistance (C2PRI) introduced in [9]. Informally, a KEM satisfies C2PRI if it is computationally hard for any adversary, even given the secret decapsulation key, to find a distinct ciphertext that decapsulates to the same session key for a given ciphertext.

Definition 10 (C2PRI Security for KEM [9]). *The* C2PRI *experiment is defined in Fig. 4. The* C2PRI *advantage of an adversary* $\mathcal{A}$ *against a KEM scheme* KEM *is*

$$\mathsf{Adv}^{\mathsf{C2PRI}}_{\mathsf{KEM}}(\mathcal{A}) := \Pr\left[\mathsf{C2PRI}^{\mathcal{A}}_{\mathsf{KEM}} = 1\right].$$

$\mathsf{C2PRI}^{\mathcal{A}}_{\mathsf{KEM}}$

1 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KeyGen}()$
2 : $(c^*, k^*) \leftarrow\!\!\$\ \mathsf{Encaps}(\mathsf{pk})$
3 : $c \leftarrow \mathcal{A}(\mathsf{pk}, \mathsf{sk}, c^*, k^*)$
4 : **if** $c \neq c^* \wedge \mathsf{Decaps}(\mathsf{sk}, c) = k^*$
5 : **return** 1
6 : **return** 0

Fig. 4: The C2PRI security game for KEM $\mathsf{KEM} = (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$.

This property may hold even for KEMs that are not IND-CCA secure. Intuitively, revealing the secret decapsulation key to the adversary yields a conservative formulation of the notion. The C2PRI notion is also related to *key-binding* properties studied by Cremers et al. [18] and Alwen et al. [8]. In particular,

C2PRI is strictly weaker than the M-BIND-K-CT notion of Cremers et al. It was shown in [7, 16] that KEMs obtained from a certain class of Fujisaki–Okamoto transformations satisfy C2PRI in both the random oracle model (ROM) [11] and the quantum random oracle model (QROM) [14]. This includes post-quantum schemes such as ML-KEM, (e)FrodoKEM, HQC, Classic McEliece, and various NTRU variants.

We now introduce *partial ciphertext second-preimage resistance* (PC2PRI) for public-key encryption schemes. Let ciphertexts be elements of a ciphertext space $\mathcal{C}$, and let $\Pi : \mathcal{C} \to \mathcal{C}_p \times \mathcal{C}_q$ be a fixed public partition function. For $c \in \mathcal{C}$, we write $(c_p, c_q) := \Pi(c)$. For example, in an ECIES-style encryption scheme with ciphertext $c = (R, C, t)$, one may define $\Pi(c) = (t, (R, C))$ or $\Pi(c) = ((R, C), t)$, depending on which component is fixed in the PC2PRI experiment. When the partition is clear from context, we directly write $(c_p, c_q) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m)$ to denote that $c \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m)$ and $(c_p, c_q) := \Pi(c)$. Similarly, we write $m = \mathsf{Dec}(\mathsf{sk}, (c_p, c_q))$.

In the PC2PRI experiment (Fig. 5), the adversary is given the secret decryption key together with a challenge ciphertext $(c_p^*, c_q^*) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m^*)$, where m^* is sampled uniformly from the message space. The adversary succeeds if it outputs $c_q' \in \mathcal{C}_q$ with $c_q' \neq c_q^*$ such that $\mathsf{Dec}(\mathsf{sk}, (c_p^*, c_q')) = m^*$. This property may still hold even if the underlying PKE scheme is not IND-CCA secure. Moreover, if a PKE scheme satisfies c_{p_1}-PC2PRI, then it also satisfies (c_{p_1}, c_{p_2})-PC2PRI.

Definition 11 (PC2PRI Security for PKE). *Let* $\mathsf{PKE} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *be a public-key encryption scheme with ciphertext space* $\mathcal{C}$*, and let* $\Pi : \mathcal{C} \to \mathcal{C}_p \times \mathcal{C}_q$ *be a public partition function. The experiment* c_p-$\mathsf{PC2PRI}^{\mathcal{A}}_{\mathsf{PKE}}$ *is defined in Fig. 5. The advantage of an adversary* $\mathcal{A}$ *is*

$$\mathsf{Adv}^{c_p\text{-}\mathsf{PC2PRI}}_{\mathsf{PKE}}(\mathcal{A}) := \Pr\left[c_p\text{-}\mathsf{PC2PRI}^{\mathcal{A}}_{\mathsf{PKE}} = 1\right].$$

c_p-$\mathsf{PC2PRI}^{\mathcal{A}}_{\mathsf{PKE}}$

1 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\$\ \mathsf{KeyGen}()$
2 : $m^* \leftarrow\!\$\ \mathcal{M}$
3 : $(c_p^*, c_q^*) \leftarrow \mathsf{Enc}(\mathsf{pk}, m^*)$
4 : $c_q' \leftarrow \mathcal{A}(\mathsf{pk}, \mathsf{sk}, m^*, c_p^*, c_q^*)$
5 : **if** $c_q' \neq c_q^* \wedge \mathsf{Dec}(\mathsf{sk}, (c_p^*, c_q')) = m^*$
6 : **return** 1
7 : **return** 0

Fig. 5: The c_p-PC2PRI game for a PKE $\mathsf{PKE} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$.

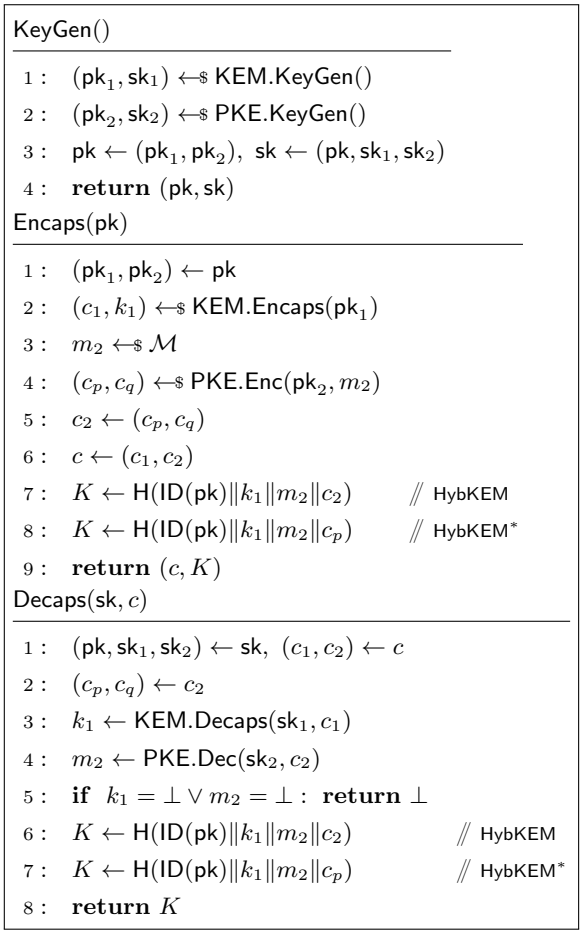

Fig. 6: Hybrid KEM constructions HybKEM and HybKEM*.

3 Hybrid KEM from PQ KEMs and Classical PKEs

In this section we present two generic hybrid KEM constructions that combine a classical PKE scheme with a post-quantum KEM satisfying the C2PRI property, as shown in Fig. 6.

Let the ciphertext space of the PKE scheme be $\mathcal{C}$. We fix a public projection function $\Pi : \mathcal{C} \to \mathcal{C}_p \times \mathcal{C}_q$. For any ciphertext $c \in \mathcal{C}$ we write $(c_p, c_q) := \Pi(c)$. For simplicity, we write $(c_p, c_q) \leftarrow\!\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}, m)$ to denote that $c \leftarrow\!\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}, m)$ and $(c_p, c_q) := \Pi(c)$. Similarly, we write $m = \mathsf{PKE.Dec}(\mathsf{sk}, (c_p, c_q))$. Both constructions derive the shared key as $\mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1 \| m_2 \| \cdot)$, where $\mathsf{ID} : \mathcal{PK} \to \{0,1\}^\gamma$ is a deterministic encoding with min-entropy $\ell := H_\infty(\mathsf{ID}(\mathsf{pk}))$. Including $\mathsf{ID}(\mathsf{pk})$ provides user-wise domain sep-

aration and enables tighter reductions in the multi-user setting, following the $\mathsf{FO}_{\mathsf{ID}(\mathsf{pk}),m}$ paradigm [21]. In practice, one may simply set $\mathsf{ID}(\mathsf{pk}) = \mathsf{pk}_2$.

The first construction, HybKEM, derives the shared key from the full PKE ciphertext c_2, namely $\mathsf{H}(\mathsf{ID}(\mathsf{pk})\|k_1\|m_2\|c_2)$. It achieves post-quantum IND-CCA security assuming that the PQ KEM is IND-CCA secure and that H is a secure PRF. Moreover, it achieves classical IND-CCA security if the classical PKE is IND-CCA secure, the PQ KEM satisfies C2PRI, and H is a secure PRF. Formal analyses are given in Theorems 1 and 2.

The second construction, HybKEM*, derives the shared key only from the projected ciphertext component c_p, namely $\mathsf{H}(\mathsf{ID}(\mathsf{pk})\|k_1\|m_2\|c_p)$. It achieves post-quantum IND-CCA security if the PQ KEM is IND-CCA secure, the classical PKE satisfies the c_p-PC2PRI property with respect to the projection Π, and H is a secure PRF. It further achieves classical IND-CCA security if the classical PKE is IND-CCA secure, the PQ KEM satisfies C2PRI, and H is a secure PRF. Formal analyses are given in Theorems 3 and 4.

In particular, the analyses for Theorems 1, 2, 3, and 4 are carried out in the standard model.

Theorem 1 (PQ IND-CCA security of HybKEM). *Assume that the underlying PQ* KEM *is δ-correct and* IND-CCA *secure, and that* H *is a secure* PRF *with output length n when keyed on k_1. Then* HybKEM *is* IND-CCA *secure. Specifically, for any QPT adversary $\mathcal{A}$ against the* IND-CCA *security of* HybKEM *making at most q_D decapsulation queries, there exist QPT adversaries $\mathcal{B}$ and $\mathcal{C}$ such that*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}}(\mathcal{A}) \le 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}) + \delta,$$

where $\mathcal{B}$ and $\mathcal{C}$ have running time approximately that of $\mathcal{A}$, and $\mathcal{B}$ makes at most q_D queries to its own decapsulation oracle.

Proof. Let $\mathcal{A}$ be an adversary against the IND-CCA security of HybKEM. Consider the games in Fig. 7.

Game G_0. This is the standard IND-CCA security game for HybKEM. Therefore,

$$\left|\Pr\left[G_0^{\mathcal{A}} \Rightarrow 1\right] - \frac{1}{2}\right| = \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}}(\mathcal{A}).$$

Game G_1. In this game, the challenge shared key k_1^* generated by the KEM is replaced with a uniformly random key sampled from $\mathcal{K}$. Furthermore, in the simulation of the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$, if the corresponding KEM ciphertext c_1 equals the challenge KEM ciphertext c_1^*, the decapsulated key is set to $k_1 = k_1^*$. For any adversary $\mathcal{A}$ that distinguishes between G_0 and G_1 with non-negligible advantage, we construct an efficient adversary $\mathcal{B}$ that attacks the IND-CCA security of KEM as follows.

Specifically, we construct the adversary $\mathcal{B}$ as described in Fig. 8. The adversary $\mathcal{B}$ receives its KEM challenge tuple $(\mathsf{pk}_1, k^*_{1\hat{b}}, c_1^*)$, where $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow\!\$\ \mathsf{KEM.KeyGen}()$, $(k^*_{10}, c_1^*) \leftarrow\!\$\ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$, $k^*_{11} \leftarrow\!\$\ \mathcal{K}$, and $\hat{b} \leftarrow\!\$\ \{0,1\}$. $\mathcal{B}(\mathsf{pk}_1, k^*_{1\hat{b}}, c_1^*)$ simulates the PKE part of the game by generating a key pair

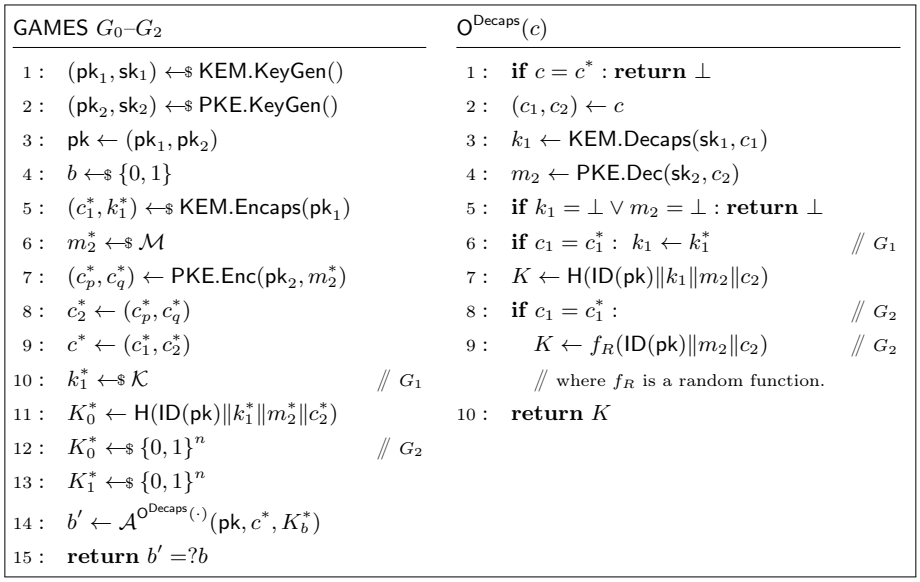

Fig. 7: Games G_0–G_2 in the proof of Theorem 1.

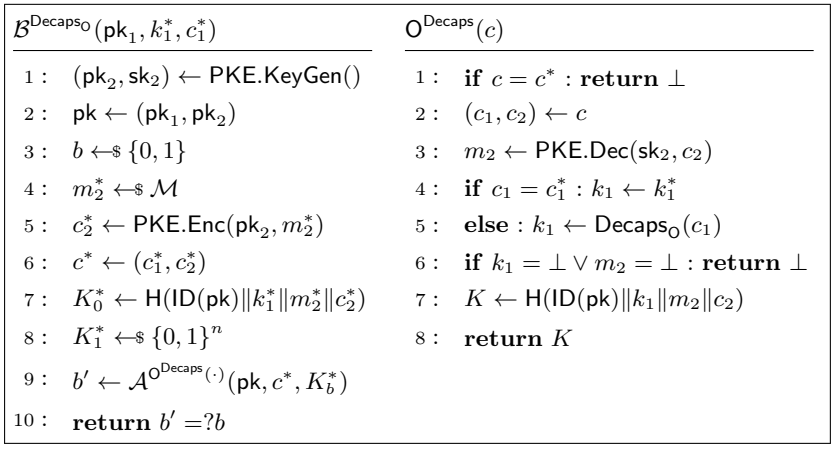

Fig. 8: Adversary $\mathcal{B}$ against the IND-CCA security of the underlying KEM. The adversary $\mathcal{B}$ makes at most q_D queries to $\mathsf{Decaps}_\mathsf{O}$.

$(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{PKE.KeyGen}()$, sampling a random message $m_2^* \leftarrow\!\!\$\ \mathcal{M}$, and computing $c_2^* = \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$. Next, $\mathcal{B}$ sets $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, $c^* = (c_1^*, c_2^*)$, and computes $K_0^* = \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_{1\hat{b}}^* \| m_2^* \| c_2^*)$. It then samples $b \leftarrow\!\!\$\ \{0,1\}$ and $K_1^* \leftarrow\!\!\$\ \{0,1\}^n$, and invokes $\mathcal{A}(\mathsf{pk}, c^*, K_b^*)$. The decapsulation oracle is simulated by $\mathcal{B}$ as follows: for queries involving ciphertexts $c = (c_1, c_2)$, $\mathcal{B}$ computes the PKE part using sk_2, and uses its own KEM decapsulation oracle for all ciphertexts except c_1^*. If $c_1 = c_1^*$, $\mathcal{B}$ uses the challenge key $k_{1\hat{b}}^*$ to simulate the response. When $\hat{b} = 0$, $\mathcal{B}$ perfectly simulates game G_0, except with probability bounded by the δ-correctness error of the KEM. When $\hat{b} = 1$, $\mathcal{B}$ perfectly simulates game G_1. Hence, we have $\left|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathcal{B} \Rightarrow 1 : \hat{b} = 0]\right| \leq \delta$, and $\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathcal{B} \Rightarrow 1 : \hat{b} = 1]$. Moreover, $\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) = \frac{1}{2}\left|\Pr[\mathcal{B} \Rightarrow 1 : \hat{b} = 0] - \Pr[\mathcal{B} \Rightarrow 1 : \hat{b} = 1]\right|$. Therefore,

$$\left|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]\right| \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) + \delta.$$

$\mathcal{C}^{\mathsf{Eval}_{\mathsf{H}}(\cdot)}$

1 : $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{KEM.KeyGen}()$
2 : $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{PKE.KeyGen}()$
3 : $\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$
4 : $b \leftarrow\!\!\$\ \{0,1\}$
5 : $(c_1^*, k_1^*) \leftarrow\!\!\$\ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$
6 : $m_2^* \leftarrow\!\!\$\ \mathcal{M}$
7 : $c_2^* \leftarrow\!\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$
8 : $c^* \leftarrow (c_1^*, c_2^*)$
9 : $K_0^* \leftarrow \mathsf{Eval}_{\mathsf{H}}(\mathsf{ID}(\mathsf{pk}) \| m_2^* \| c_2^*)$
10 : $K_1^* \leftarrow\!\!\$\ \{0,1\}^n$
11 : $b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Decaps}}(\cdot)}(\mathsf{pk}, c^*, K_b^*)$
12 : **return** $b' =? b$

$\mathsf{O}^{\mathsf{Decaps}}(c)$

1 : **if** $c = c^*$ **return** $\perp$
2 : $(c_1, c_2) \leftarrow c$
3 : $m_2 \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_2, c_2)$
4 : $k_1 \leftarrow \mathsf{KEM.Decaps}(\mathsf{sk}_1, c_1)$
5 : **if** $k_1 = \perp \vee m_2 = \perp$ **return** $\perp$
6 : **if** $c_1 = c_1^*$
7 : $\quad K \leftarrow \mathsf{Eval}_{\mathsf{H}}(\mathsf{ID}(\mathsf{pk}) \| m_2 \| c_2)$
8 : $\quad$ **return** K
9 : $K \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1 \| m_2 \| c_2)$
10 : **return** K

Fig. 9: Adversary $\mathcal{C}$ against the PRF security of H.

Game G_2. In this game, we replace the corresponding keys derived using k_1^* with independent uniformly random values. In particular, this includes the challenge key K_0^* and the shared keys returned by the decapsulation oracle whenever the KEM component satisfies $c_1 = c_1^*$. For any adversary $\mathcal{A}$ that distinguishes between G_1 and G_2, we construct an adversary $\mathcal{C}$ against the PRF security of H. Specifically, we construct $\mathcal{C}$ as shown in Fig. 9. That is, $\mathcal{C}$ perfectly simulates game G_1, except that it uses its oracle $\mathsf{Eval}_{\mathsf{H}}$ to compute the keys derived from k_1^*. If the oracle $\mathsf{Eval}_{\mathsf{H}}$ implements a real PRF, then $\mathcal{C}$ perfectly simulates G_1. If it

implements a random function, then $\mathcal{C}$ perfectly simulates G_2. Hence, $\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathsf{PRF}_0^{\mathsf{H}}(\mathcal{C}) \Rightarrow 1]$, $\Pr[G_2^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathsf{PRF}_1^{\mathsf{H}}(\mathcal{C}) \Rightarrow 1]$. Therefore,

$$\left|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}_{\mathsf{H}}^{\mathsf{PRF}}(\mathcal{C}).$$

In game G_2, the bit b is independent of the adversary's view. Thus,

$$\Pr[G_2^{\mathcal{A}} \Rightarrow 1] = \frac{1}{2}.$$

Putting everything together,

$$\mathsf{Adv}_{\mathsf{HybKEM}}^{\mathsf{IND\text{-}CCA}}(\mathcal{A}) \leq 2\,\mathsf{Adv}_{\mathsf{KEM}}^{\mathsf{IND\text{-}CCA}}(\mathcal{B}) + \mathsf{Adv}_{\mathsf{H}}^{\mathsf{PRF}}(\mathcal{C}) + \delta.$$

□

Theorem 2 (Classical IND-CCA security of HybKEM). *Assume that the underlying* PKE *is perfectly correct and* IND-CCA *secure, the* KEM *is* C2PRI*-secure, and that* H *is a secure* PRF *with output length n when keyed on m_2. Then* HybKEM *is* IND-CCA *secure. More precisely, for any PPT adversary $\mathcal{A}$ against the* IND-CCA *security of* HybKEM *making at most q_D decapsulation queries, there exist PPT adversaries $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ such that*

$$\mathsf{Adv}_{\mathsf{HybKEM}}^{\mathsf{IND\text{-}CCA}}(\mathcal{A}) \leq 2\,\mathsf{Adv}_{\mathsf{PKE}}^{\mathsf{IND\text{-}CCA}}(\mathcal{B}) + \mathsf{Adv}_{\mathsf{H}}^{\mathsf{PRF}}(\mathcal{C}) + \mathsf{Adv}_{\mathsf{KEM}}^{\mathsf{C2PRI}}(\mathcal{D}),$$

where $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ run in time approximately that of $\mathcal{A}$, and $\mathcal{B}$ makes at most q_D queries to its own decryption oracle.

Proof. Let $\mathcal{A}$ be an adversary against the IND-CCA security of HybKEM. Consider the games in Fig. 10.

Game G_0. This is the standard IND-CCA security game for HybKEM. Therefore,

$$\left|\Pr\left[G_0^{\mathcal{A}} \Rightarrow 1\right] - \frac{1}{2}\right| = \mathsf{Adv}_{\mathsf{HybKEM}}^{\mathsf{IND\text{-}CCA}}(\mathcal{A}).$$

Game G_1. In this game, we use an independent uniformly random message $m_2'^*$ to compute K_0^*. Moreover, in the simulation of the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$, whenever $c_2 = c_2^*$, we set $m_2 := m_2'^*$. For any adversary $\mathcal{A}$ that distinguishes between G_0 and G_1, we construct an adversary $\mathcal{B}$ against the IND-CCA security of the PKE scheme.

Specifically, adversary $\mathcal{B}$ is defined as in Fig. 11. Upon receiving the public key pk_2, where $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow \mathsf{PKE.KeyGen}()$, $\mathcal{B}$ samples two messages $m_{20}^*, m_{21}^* \leftarrow\!\!\$\ \mathcal{M}$. The challenger of $\mathcal{B}$ samples a random bit $\hat{b} \leftarrow \{0,1\}$ and returns the challenge ciphertext $c_{2\hat{b}}^* \leftarrow \mathsf{PKE.Enc}(\mathsf{pk}_2, m_{2\hat{b}}^*)$. Next, $\mathcal{B}$ simulates the KEM component by generating $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow\!\!\$\ \mathsf{KEM.KeyGen}()$ and computing $(c_1^*, k_1^*) \leftarrow\!\!\$\ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$. It then sets $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, $c_2^* := c_{2\hat{b}}^*$, and $c^* := (c_1^*, c_2^*)$. Finally, $\mathcal{B}$ computes $K_0^* = H(\mathsf{ID}(\mathsf{pk}) \| k_1^* \| m_{20}^* \| c_2^*)$, samples $K_1^* \leftarrow\!\!\$\ \mathcal{K}$ and $b \leftarrow \{0,1\}$, and runs $\mathcal{A}(\mathsf{pk}, c^*, K_b^*)$. To answer decapsulation queries on input $c = (c_1, c_2)$, $\mathcal{B}$ decapsulates the KEM component using sk_1, and uses its own PKE

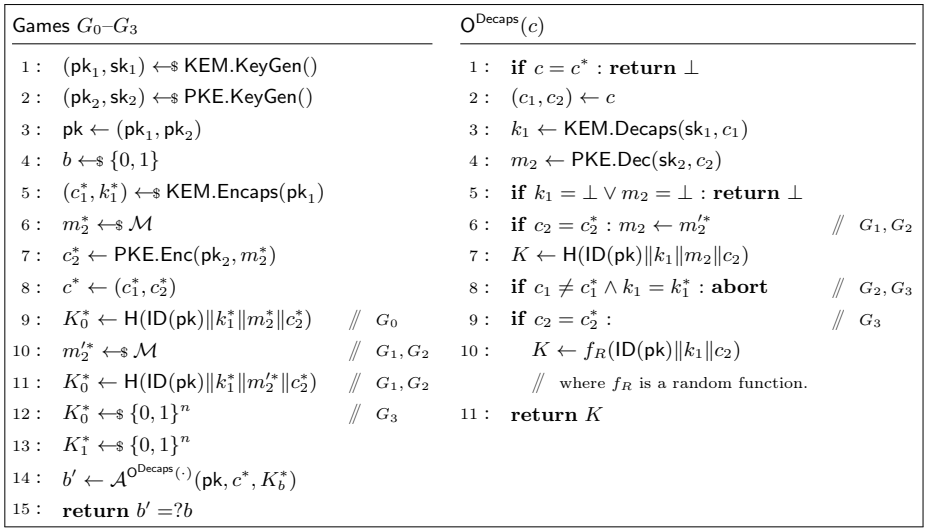

Fig. 10: Games G_0–G_3 used in the proof of Theorem 2.

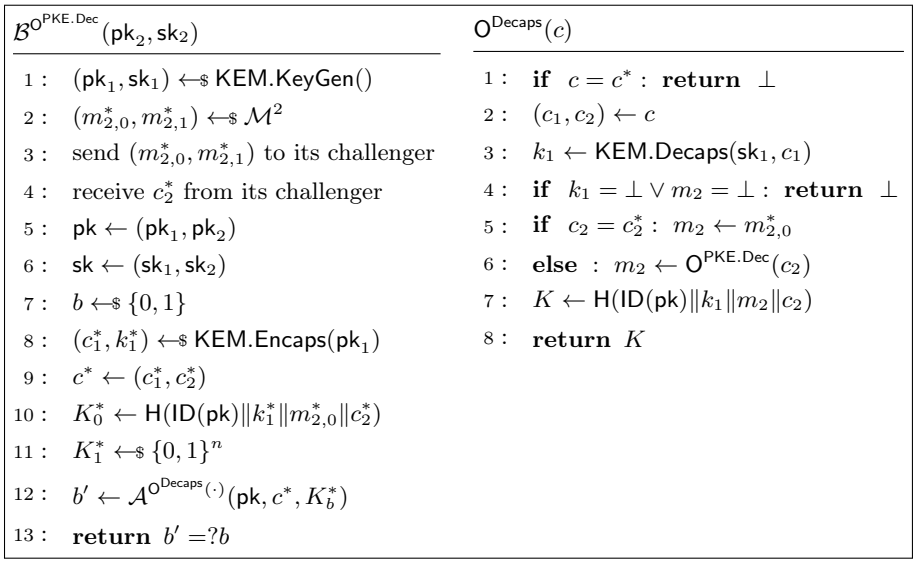

Fig. 11: Adversary $\mathcal{B}$ against the IND-CCA security of the underlying PKE. The adversary $\mathcal{B}$ makes at most q_D queries to its decryption oracle.

decryption oracle for all ciphertexts except c_2^*. If $c_2 = c_2^*$, then $\mathcal{B}$ returns m_{20}^* as the decryption result.

When $\hat{b} = 0$, $\mathcal{B}$ perfectly simulates game G_0. When $\hat{b} = 1$, $\mathcal{B}$ perfectly simulates game G_1. Hence, we have $\Pr[G_0^{\mathcal{A}} \Rightarrow 1] = \Pr[G^0_{\mathsf{PKE},\mathcal{B}} \Rightarrow 1]$ and $\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G^1_{\mathsf{PKE},\mathcal{B}} \Rightarrow 1]$. Therefore,

$$\left|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]\right| \leq 2\ \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{PKE}}(\mathcal{B}).$$

$\mathcal{D}(\mathsf{pk}_1, \mathsf{sk}_1, k_1^*, c_1^*)$	$\mathsf{O}^{\mathsf{Decaps}}(c)$
$(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow\!\!\$\ \mathsf{PKE.KeyGen}()$	**if** $c = c^*$: **return** $\perp$
$\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$	$(c_1, c_2) \leftarrow c$
$b \leftarrow\!\!\$\ \{0,1\}$	$k_1 \leftarrow \mathsf{KEM.Decaps}(\mathsf{sk}_1, c_1)$
$m_2^* \leftarrow\!\!\$\ \mathcal{M}$	$m_2 \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_2, c_2)$
$c_2^* \leftarrow\!\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$	**if** $k_1 = \perp \vee m_2 = \perp$: **return** $\perp$
$c^* \leftarrow (c_1^*, c_2^*)$	**if** $c_2 = c_2^*$: $m_2 \leftarrow m_2'^*$
$m_2'^* \leftarrow\!\!\$\ \mathcal{M}$	$K \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1 \| m_2 \| c_2)$
$K_0^* \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1^* \| m_2'^* \| c_2^*)$	**if** $c_1 \neq c_1^* \wedge k_1 = k_1^*$:
$K_1^* \leftarrow\!\!\$\ \{0,1\}^n$	**stop with** c_1
$b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Decaps}}(\cdot)}(\mathsf{pk}, c^*, K_b^*)$	**return** K
return $b' =? b$	

Fig. 12: Adversary $\mathcal{D}$ against the C2PRI security of the underlying KEM.

Game G_2. In this game, we immediately abort if the adversary queries the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$ with a ciphertext $c_1 \neq c_1^*$ such that the resulting shared key satisfies $k_1 = k_1^*$. Whenever game G_2 aborts, we can construct an adversary $\mathcal{D}$ against the C2PRI security of the KEM scheme, as illustrated in Fig. 12. The adversary $\mathcal{D}$ perfectly simulates game G_1 using the secret key provided by its challenger and embeds the challenge pair $(\mathsf{pk}_1, \mathsf{sk}_1, k_1^*, c_1^*)$ into the simulation. Upon an abort event, $\mathcal{D}$ outputs the corresponding ciphertext $c_1 \neq c_1^*$ that decapsulates to the same shared key. Such a ciphertext constitutes a valid solution to the C2PRI game. Since games G_1 and G_2 are identical from the adversary's view whenever the abort event does not occur, we obtain

$$\left|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{C2PRI}}_{\mathsf{KEM}}(\mathcal{D}).$$

Game G_3. In this game, we replace all keys derived using $m_2'^*$ with independent uniformly random values. In particular, this includes the challenge key K_0^* and the shared keys returned by the decapsulation oracle whenever the PKE component satisfies $c_2 = c_2^*$. For any adversary $\mathcal{A}$ distinguishing between G_2 and G_3,

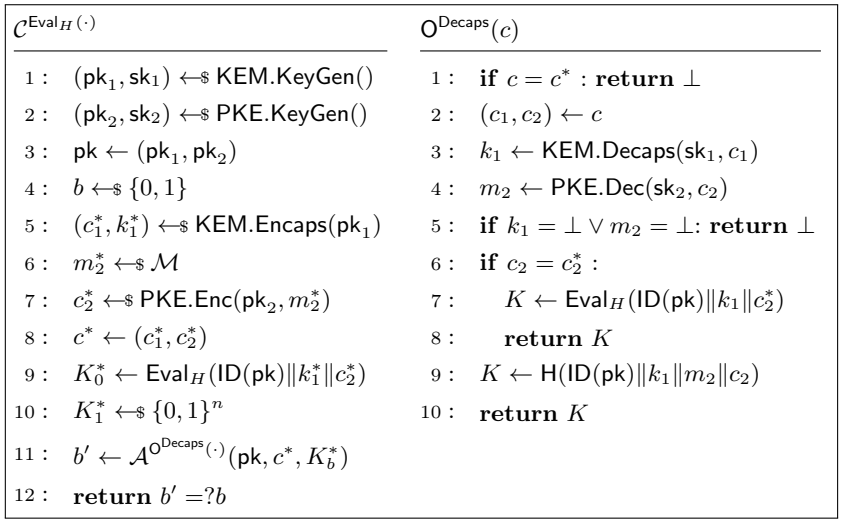

Fig. 13: Adversary $\mathcal{C}$ against the PRF security of H.

we construct an adversary $\mathcal{C}$ against the PRF security of H, with running time comparable to that of $\mathcal{A}$.

Specifically, we construct $\mathcal{C}$ as shown in Fig. 13. The adversary $\mathcal{C}$ perfectly simulates game G_2 for $\mathcal{A}$, except that it uses its oracle Eval_H to compute the challenge key K_0^* and the key K in the decapsulation oracle whenever $c_2 = c_2^*$. If the oracle Eval_H implements a real PRF, then $\mathcal{C}$ perfectly simulates game G_2. When Eval_H implements a truly random function, due to the modification in game G_2, whenever $c_1 \neq c_1^*$, we must have $k_1 \neq k_1^*$ when the game does not abort. This ensures that the random challenge key K_0^* remains independent of all keys returned by the decapsulation oracle. Therefore, $\mathcal{C}$ perfectly simulates game G_3. Hence, $\Pr[G_2^\mathcal{A} \Rightarrow 1] = \Pr[G_{\mathsf{H},\mathcal{C}}^0 \Rightarrow 1]$ and $\Pr[G_3^\mathcal{A} \Rightarrow 1] = \Pr[G_{\mathsf{H},\mathcal{C}}^1 \Rightarrow 1]$. Therefore,

$$\left|\Pr[G_2^\mathcal{A} \Rightarrow 1] - \Pr[G_3^\mathcal{A} \Rightarrow 1]\right| \leq \mathsf{Adv}_\mathsf{H}^\mathsf{PRF}(\mathcal{C}).$$

In game G_3, the bit b is independent of the adversary's view. Thus,

$$\Pr[G_3^\mathcal{A} \Rightarrow 1] = \frac{1}{2}.$$

Putting the bounds together, we obtain

$$\mathsf{Adv}_{\mathsf{HybKEM}}^{\mathsf{IND\text{-}CCA}}(\mathcal{A}) \leq 2\ \mathsf{Adv}_{\mathsf{PKE}}^{\mathsf{IND\text{-}CCA}}(\mathcal{B}) + \mathsf{Adv}_\mathsf{H}^\mathsf{PRF}(\mathcal{C}) + \mathsf{Adv}_{\mathsf{KEM}}^{\mathsf{C2PRI}}(\mathcal{D}).$$

□

Theorem 3 (PQ IND-CCA security of HybKEM^*). *Assume that the PQ* KEM *is* δ*-correct and* $\mathsf{IND\text{-}CCA}$ *secure, that the* PKE *is* c_p-$\mathsf{PC2PRI}$*-secure, and that* H *is a secure* PRF *with output length* n *when keyed on* k_1*. Then* HybKEM^* *is*

IND-CCA *secure. More precisely, for any QPT adversary* $\mathcal{A}$ *against the* IND-CCA *security of* HybKEM* *making at most* q_D *decapsulation queries, there exist QPT adversaries* $\mathcal{B}$, $\mathcal{C}$, *and* $\mathcal{D}$ *such that*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}) \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}) + \mathsf{Adv}^{c_p\text{-}\mathsf{PC2PRI}}_{\mathsf{PKE}}(\mathcal{D}) + \delta,$$

where $\mathcal{B}$, $\mathcal{C}$, *and* $\mathcal{D}$ *run in time approximately that of* $\mathcal{A}$, *and* $\mathcal{B}$ *makes at most* q_D *queries to its own decapsulation oracle.*

Proof. Let $\mathcal{A}$ be an adversary against the IND-CCA security of HybKEM*. Consider the sequence of games shown in Fig. 14.

Games G_0–G_3		$\mathsf{O}^{\mathsf{Decaps}}(c)$	
$(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow\!\$\ \mathsf{KEM.KeyGen}()$		**if** $c = c^*$: **return** $\perp$	
$(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow\!\$\ \mathsf{PKE.KeyGen}()$		$(c_1, c_2) \leftarrow c$	
$\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$		$(c_p, c_q) \leftarrow c_2$	
$b \leftarrow\!\$\ \{0,1\}$		$k_1 \leftarrow \mathsf{KEM.Decaps}(\mathsf{sk}_1, c_1)$	
$(c_1^*, k_1^*) \leftarrow\!\$\ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$		$m_2 \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_2, c_2)$	
$m_2^* \leftarrow\!\$\ \mathcal{M}$		**if** $k_1 = \perp \vee m_2 = \perp$: **return** $\perp$	
$(c_p^*, c_q^*) \leftarrow\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$		**if** $c_1 = c_1^*$: $k_1 \leftarrow k_1^*$	// G_1, G_2
$c_2^* \leftarrow (c_p^*, c_q^*)$		**if** $(c_p = c_p^*) \wedge (c_q \neq c_q^*) \wedge (m_2 = m_2^*)$:	// G_2, G_3
$c^* \leftarrow (c_1^*, c_2^*)$		**abort**	// G_2, G_3
$k_1^* \leftarrow\!\$\ \mathcal{K}$	// G_1, G_2	$K \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1 \| m_2 \| c_p)$	
$K_0^* \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1^* \| m_2^* \| c_p^*)$		**if** $c_1 = c_1^*$:	// G_3
$K_0^* \leftarrow\!\$\ \{0,1\}^n$	// G_3	$K \leftarrow f_R(\mathsf{ID}(\mathsf{pk}) \| m_2 \| c_p)$	
$K_1^* \leftarrow\!\$\ \{0,1\}^n$		// where f_R is a random function.	
$b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Decaps}}(\cdot)}(\mathsf{pk}, c^*, K_b^*)$		**return** K	
return $b' =? b$			

Fig. 14: Games G_0–G_3 for the proof of Theorem 3.

Game G_0**.** This is the standard IND-CCA experiment for HybKEM*. Hence,

$$\left|\Pr\left[G_0^{\mathcal{A}} \Rightarrow 1\right] - \tfrac{1}{2}\right| = \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}).$$

Game G_1**.** In this game, the challenge KEM key k_1^* is replaced by an independent uniformly random value. Moreover, in the simulation of the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$, whenever a query contains $c_1 = c_1^*$, we set the decapsulated key to $k_1 := k_1^*$. For any adversary $\mathcal{A}$ that distinguishes between games G_0 and G_1, we construct an adversary $\mathcal{B}$ against the IND-CCA security of the underlying KEM.

Specifically, the construction of $\mathcal{B}$ is analogous to that used in the proof of Game G_1 in Theorem 1. The adversary $\mathcal{B}$ receives the challenge tuple $(\mathsf{pk}_1, k_{1\hat{b}}^*, c_1^*)$, generates $(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow\!\$\ \mathsf{PKE.KeyGen}()$, samples $m_2^* \leftarrow\!\$\ \mathcal{M}$, and computes $(c_p^*, c_q^*) \leftarrow\!\$\ \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$. It then sets $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$ and $c^* = (c_1^*, (c_p^*, c_q^*))$, and defines $K_0^* = \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_{1\hat{b}}^* \| m_2^* \| c_p^*)$. Next, it samples

$b \leftarrow\$ \{0,1\}$ and $K_1^* \leftarrow\$ \{0,1\}^n$, and runs $\mathcal{A}(\mathsf{pk}, c^*, K_b^*)$. The decapsulation oracle is simulated using sk_2 together with the KEM decapsulation oracle, except that when $c_1 = c_1^*$, the value $k_{1\hat{b}}^*$ is directly used as the decapsulated key.

If $\hat{b} = 0$, $\mathcal{B}$ perfectly simulates game G_0, except with probability at most δ due to possible KEM decapsulation failure. If $\hat{b} = 1$, $\mathcal{B}$ perfectly simulates game G_1. Therefore,

$$\left|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]\right| \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) + \delta.$$

Game G_2. In this game, we abort whenever $\mathcal{A}$ queries $\mathsf{O}^{\mathsf{Decaps}}$ on a ciphertext $c_2 = (c_p, c_q)$ such that $c_p = c_p^*$, $c_q \neq c_q^*$, and the decrypted message satisfies $m_2 = m_2^*$. Such an abort event immediately yields an adversary $\mathcal{D}$ against the c_p-$\mathsf{PC2PRI}$ security of the PKE scheme. The adversary $\mathcal{D}$ perfectly simulates game for $\mathcal{A}$ as G_1 except it use its challenge tuple $(\mathsf{pk}_2, \mathsf{sk}_2, m_2^*, (c_p^*, c_q^*))$. Whenever the abort condition is triggered, $\mathcal{D}$ outputs the corresponding ciphertext c_q satisfying $c_q \neq c_q^*$, and $\mathsf{PKE.Dec}(\mathsf{sk}_2, (c_p, c_q)) = m_2^*$. It is easy to see that $\mathcal{D}$ wins the c_p-$\mathsf{PC2PRI}$ game whenever the abort event occurs. Hence,

$$\left|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{c_p\text{-}\mathsf{PC2PRI}}_{\mathsf{PKE}}(\mathcal{D}).$$

Game G_3. In this game, all keys derived from k_1^* are replaced by independent uniformly random values. In particular, this includes the challenge key K_0^* and the shared keys returned by the decapsulation oracle whenever the KEM component satisfies $c_1 = c_1^*$. Any distinguisher between games G_2 and G_3 immediately yields an adversary $\mathcal{C}$ breaking the PRF security of H.

Specifically, the adversary $\mathcal{C}$ is given oracle access to $\mathsf{Eval}_{\mathsf{H}}$ and perfectly simulates game G_2 for $\mathcal{A}$, except that it uses its oracle to compute $K_0^* = \mathsf{Eval}_{\mathsf{H}}(\mathsf{ID}(\mathsf{pk}), m_2^*, c_p^*)$ and to compute $K = \mathsf{Eval}_{\mathsf{H}}(\mathsf{ID}(\mathsf{pk}), m_2, c_p)$ whenever $c_1 = c_1^*$ in the decapsulation oracle. If the oracle $\mathsf{Eval}_{\mathsf{H}}$ implements a real PRF, then $\mathcal{C}$ perfectly simulates game G_2. When $\mathsf{Eval}_{\mathsf{H}}$ is a truly random function, due to the modification in game G_2, any decapsulation query satisfying $c_p = c_p^*$ and $c_q \neq c_q^*$ must yield $m_2 \neq m_2^*$ (otherwise the game would abort). This guarantees that the challenge key K_0^* remains independent of all keys returned by the decapsulation oracle. Therefore, $\mathcal{C}$ perfectly simulates game G_3 if $\mathsf{Eval}_{\mathsf{H}}$ is a truly random function. Thus,

$$\left|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}).$$

In game G_3, the bit b is independent of the adversary's view. Hence,

$$\Pr[G_3^{\mathcal{A}} \Rightarrow 1] = \tfrac{1}{2}.$$

Combining all the bounds above, we have

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}) \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{KEM}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}) + \mathsf{Adv}^{c_p\text{-}\mathsf{PC2PRI}}_{\mathsf{PKE}}(\mathcal{D}) + \delta.$$

□

Theorem 4 (Classical IND-CCA security of HybKEM*). *Assume that the underlying* PKE *is perfectly correct and* IND-CCA *secure, the* KEM *is* C2PRI*-secure, and that H is a secure* PRF *with output length n when keyed on m_2. Then* HybKEM* *is* IND-CCA *secure. More precisely, for any PPT adversary $\mathcal{A}$ against the* IND-CCA *security of* HybKEM* *making at most q_D decapsulation queries, there exist PPT adversaries $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ such that*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}) \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{PKE}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}) + \mathsf{Adv}^{\mathsf{C2PRI}}_{\mathsf{KEM}}(\mathcal{D}),$$

where $\mathcal{B}$, $\mathcal{C}$, and $\mathcal{D}$ run in time approximately that of $\mathcal{A}$, and $\mathcal{B}$ makes at most q_D queries to its own decryption oracle.

Proof. The proof follows the same structure as that of Theorem 2, with the only difference being that the c_2 component is replaced by the c_p component as input to H during the computation of the key K. Consider the games in Fig. 15. We only highlight the games here.

Games G_0–G_3		$\mathsf{O}^{\mathsf{Decaps}}(c)$	
$(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow\!\$\ \mathsf{KEM.KeyGen}()$		**if** $c = c^*$: **return** $\perp$	
$(\mathsf{pk}_2, \mathsf{sk}_2) \leftarrow\!\$\ \mathsf{PKE.KeyGen}()$		$(c_1, c_2) \leftarrow c$	
$\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$		$(c_p, c_q) \leftarrow c_2$	
$b \leftarrow\!\$\ \{0,1\}$		$k_1 \leftarrow \mathsf{KEM.Decaps}(\mathsf{sk}_1, c_1)$	
$(c_1^*, k_1^*) \leftarrow\!\$\ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$		$m_2 \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_2, c_2)$	
$m_2^* \leftarrow\!\$\ \mathcal{M}$		**if** $k_1 = \perp \vee m_2 = \perp$: **return** $\perp$	
$(c_p^*, c_q^*) \leftarrow \mathsf{PKE.Enc}(\mathsf{pk}_2, m_2^*)$		**if** $c_2 = c_2^*$: $m_2 \leftarrow m_2'^*$	// G_1, G_2
$c_2^* \leftarrow (c_p^*, c_q^*)$		$K \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1 \| m_2 \| c_p)$	
$c^* \leftarrow (c_1^*, c_2^*)$		**if** $c_1 \neq c_1^* \wedge k_1 = k_1^*$: **abort**	// G_2, G_3
$K_0^* \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1^* \| m_2^* \| c_p^*)$	// G_0	**if** $c_2 = c_2^*$:	// G_3
$m_2'^* \leftarrow\!\$\ \mathcal{M}$	// G_1, G_2	$K \leftarrow f_R(\mathsf{ID}(\mathsf{pk}) \| k_1 \| c_p)$	
$K_0^* \leftarrow \mathsf{H}(\mathsf{ID}(\mathsf{pk}) \| k_1^* \| m_2'^* \| c_p^*)$	// G_1, G_2	// where f_R is a random function.	
$K_0^* \leftarrow\!\$\ \{0,1\}^n$	// G_3	**return** K	
$K_1^* \leftarrow\!\$\ \{0,1\}^n$			
$b' \leftarrow \mathcal{A}^{\mathsf{O}^{\mathsf{Decaps}}(\cdot)}(\mathsf{pk}, c^*, K_b^*)$			
return $b' =? b$			

Fig. 15: Games G_0–G_3 used in the proof of Theorem 4.

Game G_0. This is the standard IND-CCA security game for HybKEM*. Therefore,

$$\left|\Pr\left[G_0^{\mathcal{A}} \Rightarrow 1\right] - \frac{1}{2}\right| = \mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}).$$

Game G_1. In this game, we use an independent uniformly random message $m_2'^*$ to compute K_0^*. Moreover, in the simulation of the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$, whenever $c_2 = c_2^*$, we set $m_2 := m_2'^*$. For any adversary $\mathcal{A}$ that distinguishes

between G_0 and G_1, we construct an adversary $\mathcal{B}$ against the IND-CCA security of the PKE scheme, analogous to the analysis of game G_1 in Theorem 2. Thus,

$$\left|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]\right| \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{PKE}}(\mathcal{B}).$$

Game G_2. In this game, we immediately abort if the adversary queries the decapsulation oracle $\mathsf{O}^{\mathsf{Decaps}}$ with a ciphertext $c_1 \neq c_1^*$ such that the resulting shared key satisfies $k_1 = k_1^*$. Whenever game G_2 aborts, we can construct an adversary $\mathcal{D}$ against the C2PRI security of the KEM scheme, similar to the analysis of game G_2 in Theorem 2. Therefore, we obtain

$$\left|\Pr[G_1^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{C2PRI}}_{\mathsf{KEM}}(\mathcal{D}).$$

Game G_3. In this game, we replace all keys derived using $m_2'^*$ with independent uniformly random values. In particular, this includes the challenge key K_0^* and the shared keys returned by the decapsulation oracle whenever the PKE component satisfies $c_2 = c_2^*$. For any adversary $\mathcal{A}$ distinguishing between G_2 and G_3, we construct an adversary $\mathcal{C}$ against the PRF security of H, similar to the analysis of game G_3 in Theorem 2. Thus,

$$\left|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]\right| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}).$$

In game G_3, the bit b is independent of the adversary's view. Hence,

$$\Pr[G_3^{\mathcal{A}} \Rightarrow 1] = \frac{1}{2}.$$

Combining the bounds yields

$$\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{HybKEM}^*}(\mathcal{A}) \leq 2\,\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{PKE}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{H}}(\mathcal{C}) + \mathsf{Adv}^{\mathsf{C2PRI}}_{\mathsf{KEM}}(\mathcal{D}).$$

□

4 Analysis of the PC2PRI Property for Classical PKE Standards

In this section, we discuss the PC2PRI property for several standardized classical public-key encryption schemes, including ECIES, PSEC, and SM2. Notably, all these schemes achieve perfect correctness.

4.1 ECIES Encryption

The *Elliptic Curve Integrated Encryption Scheme (ECIES)* is specified in ANSI X9.63, IEEE 1363a, and ISO/IEC 18033-2 [1, 6, 25]. The ECIES scheme can be instantiated in the DHAES framework and has been proven IND-CCA secure in the standard model under the oracle Diffie–Hellman assumption and the standard security of the underlying symmetric encryption and message-authentication code [4, 5].

Algorithm 1 ECIES Public-key Encryption Scheme

Key Generation

1. Generate elliptic-curve domain parameters $D = (q, \mathbb{F}_q, S, a, b, P, n, h)$.
2. Sample $d \xleftarrow{\$} [1, n-1]$.
3. Compute $Q = [d]P$.
4. Output $(\mathsf{pk} = Q, \mathsf{sk} = d)$.

Encryption (Input: $D, \mathsf{pk} = Q, m$; Output: ciphertext $c = (R, C, t)$)

1. Sample $k \xleftarrow{\$} [1, n-1]$.
2. Compute $R = kP$ and $Z = hkQ$. If $Z = \infty$, restart.
3. Derive $(k_1, k_2) \leftarrow \mathsf{KDF}(x_Z, R)$, where x_Z is the x-coordinate of Z.
4. Compute $C \leftarrow \mathsf{ENC}_{k_1}(m)$ and $t \leftarrow \mathsf{MAC}_{k_2}(C)$.
5. Output $c = (R, C, t)$.

Decryption (Input: $D, \mathsf{sk} = d, c = (R, C, t)$; Output: m or $\perp$)

1. Validate R. If invalid, output $\perp$.
2. Compute $Z = hdR$. If $Z = \infty$, output $\perp$.
3. Derive $(k_1, k_2) \leftarrow \mathsf{KDF}(x_Z, R)$.
4. If $\mathsf{MAC}_{k_2}(C) \neq t$, output $\perp$.
5. Output $m = \mathsf{DEC}_{k_1}(C)$.

Fig. 16: The ECIES encryption scheme [24].

We only give a brief description of ECIES in Fig. 16, and the complete specification can be found in [1,6,25]. As illustrated in Fig. 16, ECIES follows a hybrid Diffie–Hellman encryption paradigm. An ephemeral Diffie–Hellman shared secret is established and then used to derive two symmetric keys k_1 and k_2 via a key-derivation function KDF. The key k_1 is used to encrypt the plaintext under a symmetric encryption scheme (ENC, DEC), while the key k_2 is used to authenticate the resulting ciphertext through a deterministic message-authentication code MAC.

PC2PRI Security for ECIES. Next, we discuss the PC2PRI property of ECIES. Recall that an ECIES ciphertext has the form $\mathsf{ct} = (R, C, t)$. We consider the following two public partition functions:

$$\Pi_t(R, C, t) := (t, (R, C)), \qquad \Pi_{R,C}(R, C, t) := ((R, C), t).$$

That is, under Π_t we have $c_p = t$ and $c_q = (R, C)$, while under $\Pi_{R,C}$ we have $c_p = (R, C)$ and $c_q = t$. We show that in both cases ECIES satisfies the corresponding c_p-PC2PRI property.

Theorem 5 (t-PC2PRI of ECIES). *With respect to the partition function $\Pi_t(R, C, t) := (t, (R, C))$, suppose that the underlying* MAC *is* MacColl*-secure and that* KDF *is modeled as a (quantum-accessible) random oracle. Then* ECIES *satisfies the t-*PC2PRI *property. More precisely, for any QPT adversary $\mathcal{A}$ making*

q_{KDF} *quantum queries to the random oracle* KDF*, there exists a QPT adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{t\text{-}\mathsf{PC2PRI}}_{\mathsf{ECIES}}(\mathcal{A}) \leq 2\,\mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{B}) + (q_{\mathsf{KDF}}+1)^3/2^{\ell_{k_2}},$$

where ℓ_{k_2} *denote the output length of the MAC key derived by* KDF*.*

Proof. Suppose that a QPT adversary $\mathcal{A}$ wins the corresponding t-$\mathsf{PC2PRI}$ game with non-negligible probability. Then $\mathcal{A}$ is given $(\mathsf{pk},\mathsf{sk}) \leftarrow\!\$\ \mathsf{KeyGen}$, $m^* \leftarrow\!\$\ \mathcal{M}$, and a challenge ciphertext $\mathsf{ct}^* = (R^*, C^*, t^*) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m^*)$. Since $c_p = t$, the adversary must output $c'_q = (R, C) \neq (R^*, C^*)$ such that $\mathsf{Dec}(\mathsf{sk}, (R, C, t^*)) = m^*$. By correctness of ECIES we have $\mathsf{MacVf}(k_2, C, t^*) = 1$ and $\mathsf{MacVf}(k_2^*, C^*, t^*) = 1$, where $k_2 = \mathsf{KDF}(x_Z, R)$ and $k_2^* = \mathsf{KDF}(x_Z^*, R^*)$ are the MAC keys derived during the corresponding decryption procedures. We distinguish two cases.

Case 1: $R = R^*$ and $C \neq C^*$. Then we have $k_2 = k_2^*$, and hence the same tag t^* is valid for two distinct messages under the same MAC key, which immediately yields a $\mathsf{MacColl}$ forgery. Thus the adversary's success probability in this case is bounded by $\mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{B})$.

Case 2: $R \neq R^*$. Consider the bad event that the random oracle KDF outputs partially collide, namely $k_2 = k_2^*$. Since KDF is modeled as a quantum random oracle, this event occurs with probability at most $(q_{\mathsf{KDF}}+1)^3/2^{\ell_{k_2}}$ by Corollary 1 of [33]. Conditioned on the complement event $k_2 \neq k_2^*$, the adversary must produce a valid tag under a fresh MAC key, which also yields a $\mathsf{MacColl}$ forgery. Thus the adversary's success probability in this case is bounded by $\mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{B}) + (q_{\mathsf{KDF}}+1)^3/2^{\ell_{k_2}}$.

Combining the two cases concludes the proof.

Theorem 6 (**(R,C)-$\mathsf{PC2PRI}$ of ECIES**)**.** *With respect to the partition function* $\Pi_{R,C}(R, C, t) := ((R, C), t)$*, suppose that the* MAC *algorithm is deterministic. Then* ECIES *satisfies the* (R, C)-$\mathsf{PC2PRI}$ *property. More precisely, for any QPT adversary* $\mathcal{A}$ *we have* $\mathsf{Adv}^{(R,C)\text{-}\mathsf{PC2PRI}}_{\mathsf{ECIES}}(\mathcal{A}) = 0$*.*

Proof. Assume that $\mathcal{A}$ wins the (R, C)-$\mathsf{PC2PRI}$ game with non-negligible probability. Then $\mathcal{A}$ is given $(\mathsf{pk},\mathsf{sk}) \leftarrow\!\$\ \mathsf{KeyGen}$, $m^* \leftarrow\!\$\ \mathcal{M}$, and a challenge ciphertext $\mathsf{ct}^* = (R^*, C^*, t^*) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m^*)$. Since $c_p = (R, C)$, the adversary must output a modified component $t' \neq t^*$ such that $\mathsf{Dec}(\mathsf{sk}, (R^*, C^*, t')) = m^*$. By correctness of ECIES, successful decryption implies that $\mathsf{MAC}(k_2^*, C^*) = t'$ and $\mathsf{MAC}(k_2^*, C^*) = t^*$. Since the MAC algorithm is deterministic, this implies $t' = t^*$, which contradicts the requirement $t' \neq t^*$. Therefore the adversary cannot win the game c_p-$\mathsf{PC2PRI}$.

4.2 PSEC

The *Provably Secure Elliptic Curve Encryption Scheme (PSEC)* is specified in ISO/IEC 18033-2 [25]. It is obtained by combining *PSEC-KEM* with a data encapsulation mechanism (DEM). Similar to ECIES, PSEC follows a hybrid

Algorithm 2 PSEC Public-key Encryption Scheme

Key Generation

1. Generate elliptic-curve domain parameters $D = (q, \mathbb{F}_q, S, a, b, P, n, h)$.
2. Sample $d \xleftarrow{\$} [1, n-1]$.
3. Compute $Q = [d]P$.
4. Output $(\mathsf{pk} = Q, \mathsf{sk} = d)$.

Encryption (Input: $D, \mathsf{pk} = Q, m$; Output: ciphertext $c = (R, C, s, t)$)

1. Sample $r \xleftarrow{\$} \{0,1\}^{\ell}$, where $\ell = \log_2 n$.
2. Compute $(k', k_1, k_2) \leftarrow \mathsf{KDF}(r)$ with $|k'| = \ell + 128$.
3. Compute $k = k' \bmod n$, $R = kP$, and $Z = kQ$.
4. Compute $s = r \oplus \mathsf{KDF}(R, Z)$.
5. Compute $C \leftarrow \mathsf{ENC}_{k_1}(m)$ and $t \leftarrow \mathsf{MAC}_{k_2}(C)$.
6. Output $c = (R, C, s, t)$.

Decryption (Input: $D, \mathsf{sk} = d, c = (R, C, s, t)$; Output: m or $\perp$)

1. Compute $Z = dR$ and $r = s \oplus \mathsf{KDF}(R, Z)$.
2. Compute $(k', k_1, k_2) \leftarrow \mathsf{KDF}(r)$ and $k = k' \bmod n$.
3. Verify $R' = kP$. If $R' \neq R$, output $\perp$.
4. If $\mathsf{MAC}_{k_2}(C) \neq t$, output $\perp$.
5. Output $m = \mathsf{DEC}_{k_1}(C)$.

Fig. 17: The PSEC encryption scheme [31].

Diffie–Hellman encryption paradigm. The PSEC scheme relies on the following cryptographic primitives: a key-derivation function KDF, a symmetric encryption scheme (ENC, DEC), and a deterministic message-authentication code algorithm MAC. The IND-CCA security of PSEC has been proven secure that the symmetric-key encryption and MAC algorithms are secure, the computational Diffie–Hellman problem, and the KDF behaves as a random function [19, 24, 25].

PC2PRI Security for PSEC. Next, we discuss the PC2PRI property of PSEC. Recall that a ciphertext of PSEC has the form $\mathsf{ct} = (R, C, s, t)$. We consider the following public partition function $\Pi_t : \mathcal{C} \to \mathcal{C}_p \times \mathcal{C}_q$ defined by $\Pi_t(R, C, s, t) := (t, (R, C, s))$. That is, under Π_t we fix the tag component $c_p = t$ and let $c_q = (R, C, s)$.

Theorem 7 (t-PC2PRI of PSEC). *With respect to the partition function Π_t, suppose that the underlying* MAC *scheme is* MacColl*-secure and that the* KDF *is modeled as a (quantum) random oracle. Then* PSEC *satisfies the t-*PC2PRI *property. More precisely, for any QPT adversary $\mathcal{A}$ that makes q_{KDF} times (quantum) queris to random oracle* KDF*, there exists a QPT adversary $\mathcal{B}$ such that*

$$\mathsf{Adv}^{t\text{-PC2PRI}}_{\mathsf{PSEC}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{B}) + (q_{\mathsf{KDF}} + 1)^3 / 2^{\ell_{k_2}},$$

where ℓ_{k_2} denote the output length of the MAC key derived by KDF*.*

Proof. Consider a QPT adversary $\mathcal{A}$ wins the t-PC2PRI game for PSEC with non-negligible probability. That is, $\mathcal{A}$ is given $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\$\ \mathsf{KeyGen}$, $m^* \leftarrow\!\$\ \mathcal{M}$, and a challenge ciphertext $\mathsf{ct}^* = (R^*, C^*, s^*, t^*) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m^*)$, and outputs $(R, C, s) \neq (R^*, C^*, s^*)$ such that $\mathsf{Dec}(\mathsf{sk}, (R, C, s, t^*)) = m^*$.

Let k_2^* and k_2 denote the MAC keys derived during the decryption of (R^*, C^*, s^*, t^*) and (R, C, s, t^*), respectively. Successful decryption implies that $\mathsf{MAC}(k_2^*, C^*) = t^*$ and $\mathsf{MAC}(k_2, C) = t^*$. We distinguish two cases.

Case 1: $k_2 \neq k_2^*$ or $C \neq C^*$. In this case we obtain two distinct MAC inputs $(k_2, C) \neq (k_2^*, C^*)$ that yield the same tag t^*. This immediately gives rise to a MacColl forgery. Hence the adversary's success probability in this case is bounded by $\mathsf{Adv}^{\mathsf{MacColl}}_{\mathsf{MAC}}(\mathcal{B})$.

Case 2: $k_2 = k_2^*$ and $C = C^*$. Let r and r^* denote the randomness values recovered during decryption from (R, C, s, t^*) and (R^*, C^*, s^*, t^*), respectively. Consider the bad event that $r \neq r^*$ while $k_2 = k_2^*$. Since KDF is modeled as a quantum-accessible random oracle, this event corresponds to a partial collision in the oracle outputs on distinct inputs and therefore occurs with probability at most $(q_{\mathsf{KDF}} + 1)^3 / 2^{\ell_{k_2}}$ by Corollary 1 of [33]. Conditioned on the bad event does not occurs, we must have $r = r^*$, which further implies $R = R^*$ and $s = s^*$. Together with $C = C^*$ this contradicts the requirement $(R, C, s) \neq (R^*, C^*, s^*)$.

Combining the above cases completes the proof.

4.3 SM2

The IND-CCA secure SM2 public-key encryption scheme is specified in the Chinese cryptographic standard GM/T 0003.4 [32]. It follows a hybrid Diffie–Hellman encryption paradigm similar to ECIES and PSEC. The protocol description for SM2 is given in Fig. 18.

PC2PRI Security for SM2. An SM2 ciphertext has the form $\mathsf{ct} = (C_1, C_2, C_3)$. Consider the public projection $\Pi_{C_1}(C_1, C_2, C_3) := (C_1, (C_2, C_3))$ and we show SM2 satisfies C_1-PC2PRI security.

Theorem 8 (C_1-PC2PRI of SM2). *With respect to the projection function Π_{C_1}, the SM2 encryption scheme satisfies the C_1-PC2PRI property. More precisely, for any QPT adversary $\mathcal{A}$, $\mathsf{Adv}^{(C_1)\text{-}\mathsf{PC2PRI}}_{\mathsf{SM2}}(\mathcal{A}) = 0$.*

Proof. In the C_1-PC2PRI experiment, the adversary $\mathcal{A}$ is given $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\$\ \mathsf{KeyGen}$, a message $m^* \leftarrow\!\$\ \mathcal{M}$, and a challenge ciphertext $\mathsf{ct}^* = (C_1^*, C_2^*, C_3^*) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{pk}, m^*)$. Then, the adversary must output a modified component $(C_2, C_3) \neq (C_2^*, C_3^*)$ such that $\mathsf{Dec}(\mathsf{sk}, (C_1^*, C_2, C_3)) = m^*$.

Fixing $C_1 = C_1^*$ uniquely determines the Diffie–Hellman shared point $S = [d]C_1^* = (x_2, y_2)$ and hence the $t = \mathsf{KDF}(x_2 \parallel y_2)$. Therefore, for the fixed message m^*, the ciphertext component $C_2 = m^* \oplus t = C_2^*$. Moreover, successful decryption requires $C_3 = H(x_2 \parallel m^* \parallel y_2)$, therefore we have $C_3 = C_3^*$. This contradicts the requirement $(C_2', C_3') \neq (C_2^*, C_3^*)$. Therefore the adversary can not win the C_1-PC2PRI game for SM2.

Algorithm 3 SM2 Public-Key Encryption Scheme [32]

Key Generation

1. Generate elliptic-curve domain parameters $D = (q, \mathbb{F}_q, S, a, b, P, n, h)$.
2. Sample $d \xleftarrow{\$} [1, n-1]$.
3. Compute $Q = [d]P$.
4. Output $(\mathsf{pk} = Q, \mathsf{sk} = d)$.

Encryption (Input: $D, \mathsf{pk} = Q, m$; Output: ciphertext (C_1, C_2, C_3))

1. Sample $k \xleftarrow{\$} [1, n-1]$.
2. Compute $C_1 = [k]P$, $S = [k]Q = (x_2, y_2)$. If $S = \infty$, output $\perp$.
3. Compute $t \leftarrow \mathsf{KDF}(x_2 \parallel y_2)$. If $t = 0^n$, restart.
4. Compute $C_2 \leftarrow m \oplus t$.
5. Compute $C_3 \leftarrow H(x_2 \parallel m \parallel y_2)$.
6. Output (C_1, C_2, C_3).

Decryption (Input: $D, \mathsf{sk} = d, c = (C_1, C_2, C_3)$; Output: m or $\perp$)

1. Validate C_1. If invalid, output $\perp$.
2. Compute shared point $S = [d]C_1 = (x_2, y_2)$. If $S = \infty$, output $\perp$.
3. Compute $t \leftarrow \mathsf{KDF}(x_2 \parallel y_2)$. If $t = 0$, output $\perp$.
4. Recover $m \leftarrow C_2 \oplus t$.
5. If $H(x_2 \parallel m \parallel y_2) \neq C_3$, output $\perp$.
6. Output m.

Fig. 18: The SM2 encryption scheme.

References

1. Ieee standard specifications for public-key cryptography - amendment 1: Additional techniques. IEEE Std 1363a-2004 (Amendment to IEEE Std 1363-2000) pp. 1–167 (2004). https://doi.org/10.1109/IEEESTD.2004.94612
2. Migration to post quantum cryptography. Tech. rep., Federal Office for Information Security (November 2020), https://www.ncsc.gov.uk/whitepaper/preparing-for-quantum-safe-cryptography
3. White paper: Next steps in preparing for post-quantum cryptography. Tech. rep., NCSC(National Cyber Security Center) (August 2024), https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography
4. Abdalla, M., Bellare, M., Rogaway, P.: DHAES: An encryption scheme based on the diffie-hellman problem. Cryptology ePrint Archive, Paper 1999/007 (1999), https://eprint.iacr.org/1999/007
5. Abdalla, M., Bellare, M., Rogaway, P.: The oracle Diffie-Hellman assumptions and an analysis of DHIES. In: Naccache, D. (ed.) Topics in Cryptology – CT-RSA 2001. Lecture Notes in Computer Science, vol. 2020, pp. 143–158. Springer Berlin Heidelberg, Germany, San Francisco, CA, USA (Apr 8–12, 2001). https://doi.org/10.1007/3-540-45353-9_12
6. Accredited Standards Committee X9: Public Key Cryptography for the Financial Services Industry: Key Agreement and Key Transport Using Elliptic Curve Cryptography (2001)

7. Alagic, G., Bajaj, F., Kocoglu, A.: The best of both KEMs: Securely combining KEMs in post-quantum hybrid schemes. Cryptology ePrint Archive, Paper 2025/1444 (2025), https://eprint.iacr.org/2025/1444
8. Alwen, J., Hartmann, D., Kiltz, E., Mularczyk, M., Schwabe, P.: Post-quantum multi-recipient public key encryption. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023: 30th Conference on Computer and Communications Security. pp. 1108–1122. ACM Press, Copenhagen, Denmark (Nov 26–30, 2023). https://doi.org/10.1145/3576915.3623185
9. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-wing. IACR Communications in Cryptology **1**(1) (2024). https://doi.org/10.62056/a3qj89n4e
10. Beguinet, H., Chevalier, C., Lebrun, G., Legavre, T., Ricosset, T., Roméas, M., Éric Sageloli: DAKE: Bandwidth-efficient (u)AKE from double-KEM. Cryptology ePrint Archive, Paper 2025/1755 (2025), https://eprint.iacr.org/2025/1755
11. Bellare, M., Rogaway, P.: Entity authentication and key distribution. In: Stinson, D.R. (ed.) Advances in Cryptology – CRYPTO'93. Lecture Notes in Computer Science, vol. 773, pp. 232–249. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 22–26, 1994). https://doi.org/10.1007/3-540-48329-2_21
12. Bellare, M., Rogaway, P.: Optimal asymmetric encryption. In: De Santis, A. (ed.) Advances in Cryptology – EUROCRYPT'94. Lecture Notes in Computer Science, vol. 950, pp. 92–111. Springer Berlin Heidelberg, Germany, Perugia, Italy (May 9–12, 1995). https://doi.org/10.1007/BFb0053428
13. Bernstein, D.J.: Curve25519: New Diffie-Hellman speed records. In: Yung, M., Dodis, Y., Kiayias, A., Malkin, T. (eds.) PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science, vol. 3958, pp. 207–228. Springer Berlin Heidelberg, Germany, New York, NY, USA (Apr 24–26, 2006). https://doi.org/10.1007/11745853_14
14. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Lee, D.H., Wang, X. (eds.) Advances in Cryptology – ASIACRYPT 2011. Lecture Notes in Computer Science, vol. 7073, pp. 41–69. Springer Berlin Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_3
15. Connolly, D., Grubbs, P.: StarFortress: Hybrid KEMs with diffie-hellman inlining. Cryptology ePrint Archive, Paper 2026/125 (2026), https://eprint.iacr.org/2026/125
16. Connolly, D., Hövelmanns, K., Hülsing, A., Kousidis, S., Meijers, M.: Starfighters — on the general applicability of x-wing. Cryptology ePrint Archive, Paper 2025/1397 (2025), https://eprint.iacr.org/2025/1397
17. Connolly, D., Ounsworth, M., Schmieg, S., Stebila, D.: StarHunters— secure hybrid post-quantum KEMs from IND-CCA2 PKEs. Cryptology ePrint Archive, Paper 2026/427 (2026), https://eprint.iacr.org/2026/427
18. Cremers, C., Dax, A., Medinger, N.: Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024: 31st Conference on Computer and Communications Security. pp. 1046–1060. ACM Press, Salt Lake City, UT, USA (Oct 14–18, 2024). https://doi.org/10.1145/3658644.3670283
19. CRYPTREC: PSEC-KEM Specification. Tech. rep., NTT Information Sharing Platform Laboratories, NTT Corporation (Apr 2008), https://www.cryptrec.go.jp/cryptrec_03_spec_cypherlist_files/PDF/02_03e_jspec.pdf

20. Diffie, W., Hellman, M.: New directions in cryptography. IEEE Transactions on Information Theory **22**(6), 644–654 (1976). https://doi.org/10.1109/TIT.1976.1055638
21. Duman, J., Hövelmanns, K., Kiltz, E., Lyubashevsky, V., Seiler, G.: Faster lattice-based KEMs via a generic fujisaki-okamoto transform using prefix hashing. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021: 28th Conference on Computer and Communications Security. pp. 2722–2737. ACM Press, Virtual Event, Republic of Korea (Nov 15–19, 2021). https://doi.org/10.1145/3460120.3484819
22. Fujisaki, E., Okamoto, T., Pointcheval, D., Stern, J.: RSA-OAEP is secure under the RSA assumption. Journal of Cryptology **17**(2), 81–104 (Mar 2004). https://doi.org/10.1007/s00145-002-0204-y
23. Giacon, F., Heuer, F., Poettering, B.: KEM combiners. In: Abdalla, M., Dahab, R. (eds.) PKC 2018: 21st International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 10769, pp. 190–218. Springer, Cham, Switzerland, Rio de Janeiro, Brazil (Mar 25–29, 2018). https://doi.org/10.1007/978-3-319-76578-5_7
24. Hankerson, D., Vanstone, S., Menezes, A.: Guide to Elliptic Curve Cryptography. Springer Professional Computing, Springer, New York (2004). https://doi.org/10.1007/b97644
25. ISO/IEC: Information technology – Security techniques – Encryption algorithms – Part 2: Asymmetric ciphers. Tech. Rep. ISO/IEC 18033-2:2006, International Organization for Standardization and International Electrotechnical Commission, Geneva, Switzerland (May 2006), https://www.iso.org/standard/37971.html, first edition, 2006-05-01
26. Langley, A., Hamburg, M., Turner, S.: Elliptic Curves for Security. RFC 7748 (Jan 2016). https://doi.org/10.17487/RFC7748
27. Liu, Y., Zhou, B., Jiang, H.: CuKEM: A Concise and Unified Hybrid Key Encapsulation Mechanism. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025). pp. 4707–4721. ACM, New York, NY, USA (2025). https://doi.org/10.1145/3719027.3744863
28. Moody, D., Perlner, R., Regenscheid, A., Robinson, A., Cooper, D.: Transition to post-quantum cryptography standards. NIST Internal Report NIST IR 8547 ipd, National Institute of Standards and Technology, Gaithersburg, MD (2024). https://doi.org/10.6028/NIST.IR.8547.ipd
29. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM **21**(2), 120–126 (1978). https://doi.org/10.1145/359340.359342
30. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science. pp. 124–134 (1994). https://doi.org/10.1109/SFCS.1994.365700
31. Standards for Efficient Cryptography Group (SECG): Sec 1: Elliptic curve cryptography, version 2.0. Tech. rep., SECG (2009), http://www.secg.org/sec1-v2.pdf
32. State Cryptography Administration of China: GM/T 0003.5–2012: Public Key Cryptographic Algorithm SM2 Based on Elliptic Curves – Part 5: Public-Key Encryption. Chinese commercial cryptography standard, State Cryptography Administration of China (2012), https://www.chinesestandard.net/PDF.aspx/GMT0003.5-2012
33. Zhandry, M.: How to record quantum queries, and applications to quantum indifferentiability. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology –

CRYPTO 2019, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 239–268. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26951-7_9